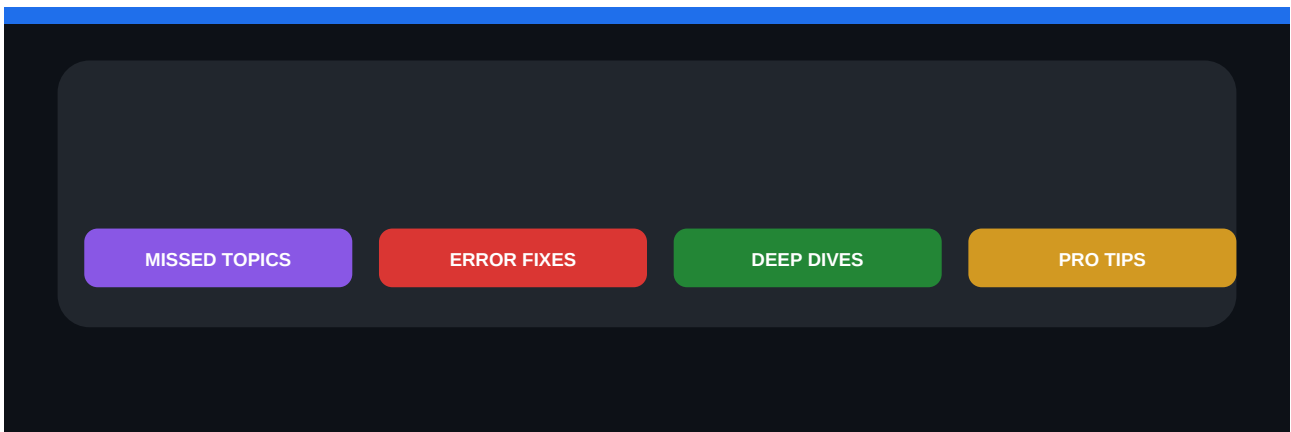




Supplement Guide

Everything not covered in the main guide — read this alongside it

Part	Topic	What's Inside
A	Git & Version Control	Track your code changes, never lose work
B	Understanding JSON APIs	How to read any API response structure
C	Django ORM Deep Dive	Queries, filtering, relationships explained
D	Static Files & CSS in Django	Making your website look good properly
E	Environment Variables & Security	Keeping secrets safe, production settings
F	Django REST Framework (API Output)	Expose your DB data as an API too
G	Logging & Monitoring	Know when things break automatically
H	Database Migrations Deep Dive	Handle schema changes without breaking data
I	Deploying to a Real Server	PythonAnywhere, Railway, or a VPS
J	FULL Error Encyclopedia	60+ errors with exact fixes and code

PART A — Git & Version Control (CRITICAL — Was Not Covered)

Git is a tool that tracks every change you make to your code. Think of it as an unlimited 'undo' button for your entire project. It is **essential** for any real project. Without it, one accidental deletion can destroy hours of work.

A1 — Install Git

- Windows: Download from <https://git-scm.com/download/win> and install
- Mac: Open Terminal and type: `xcode-select --install`
- Linux: `sudo apt install git`

```
# Verify installation
git --version
# Output: git version 2.x.x

# Set your identity (do this once)
git config --global user.name 'Your Name'
git config --global user.email 'you@example.com'
```

A2 — Initialize Git in Your Project

```
# Inside your my_api_project folder:
git init

# Create a .gitignore file to exclude secrets and junk:
# Create file named '.gitignore' and paste this inside:

venv/
__pycache__/
*.pyc
.env
db.sqlite3
*.log
.DS_Store
```

■ ■ **NOTE:** The .env file must ALWAYS be in .gitignore. Never commit passwords to Git.

A3 — Daily Git Workflow

```
# Step 1: Check what changed
git status

# Step 2: Stage your changes
git add .           # adds ALL changed files
git add settings.py # OR add one specific file

# Step 3: Save a snapshot with a message
git commit -m 'Add fetch_data command'

# Step 4: View history
git log --oneline
```

A4 — Undo Mistakes

```
# Undo changes in a file (before committing)
git checkout -- filename.py
```

```
# Go back to last commit (warning: destroys uncommitted changes)
git reset --hard HEAD
```

```
# Go back 2 commits (keeps files, undoes commits)
git reset --soft HEAD~2
```

■ **TIP:** Commit after every working change. Message should describe WHAT you did: 'Fix database connection error' not 'fix stuff'.

PART B — Understanding JSON & Your API Structure

Before you can save API data, you must understand what shape it comes in. Every API is different. Here is how to read any API response.

B1 — Inspect Your API in the Browser

- Open your API URL in Chrome/Firefox
- Install the **JSONView** browser extension (search it in your browser's extension store)
- The JSON will appear formatted and color-coded — much easier to read

B2 — Common API Response Shapes

```
# SHAPE 1: Direct list
[
  {"id": 1, "name": "Gold", "price": 1923.5},
  {"id": 2, "name": "Silver", "price": 24.1}
]
# Access: data is already a list → loop directly
records = data    # data is a list

# SHAPE 2: Wrapped in a key
{"results": [{...}, {...}], "count": 2, "next": null}
# Access:
records = data['results']

# SHAPE 3: Nested inside 'data'
{"status": "ok", "data": {"items": [{...}]}}
# Access:
records = data['data']['items']

# SHAPE 4: Single object (not a list)
{"temperature": 38.5, "city": "Dubai", "unit": "C"}
# Access: wrap in a list yourself
records = [data]
```

B3 — Debug Your API Response

```
# Add this BEFORE saving to DB – prints what you receive
import json

response = requests.get(api_url, headers=headers)
print('Status code:', response.status_code)
print('Raw response:', response.text[:500]) # first 500 chars
data = response.json()
print('Keys in response:', list(data.keys()) if isinstance(data, dict) else 'It is a list')
print('First item:', json.dumps(data[0] if isinstance(data, list) else data, indent=2))
```

B4 — Handling API Authentication Types

```
# TYPE 1: API Key in Header (most common)
headers = {'X-API-Key': os.getenv('API_KEY')}

# TYPE 2: Bearer Token
headers = {'Authorization': f'Bearer {os.getenv("API_KEY")}'}
```

```
# TYPE 3: API Key in URL query parameter
url = f'{base_url}?api_key={os.getenv("API_KEY")}'

# TYPE 4: Basic Auth (username + password)
from requests.auth import HTTPBasicAuth
response = requests.get(url, auth=HTTPBasicAuth('user', 'pass'))

# TYPE 5: No auth needed (public API)
response = requests.get(url) # no headers needed
```

■ **TIP:** If unsure which auth type your API uses, read its documentation page. Look for 'Authentication' section.

PART C — Django ORM Deep Dive (Database Queries)

The ORM (Object Relational Mapper) is how Django talks to PostgreSQL. You write Python — Django converts it to SQL. You never need to write raw SQL.

C1 — Basic CRUD Operations

```
from livedata.models import ApiRecord

# CREATE — add a new record
r = ApiRecord.objects.create(title='New Item', value=42.5)

# READ — get all records
all_records = ApiRecord.objects.all()

# READ — get one record by ID
record = ApiRecord.objects.get(id=1)

# READ — get records matching a condition
active = ApiRecord.objects.filter(is_active=True)
cheap = ApiRecord.objects.filter(value__lt=100) # value less than 100
big = ApiRecord.objects.filter(value__gte=500) # greater or equal
named = ApiRecord.objects.filter(title__contains='Gold')

# UPDATE — change a record
record = ApiRecord.objects.get(id=1)
record.value = 99.9
record.save()

# UPDATE many at once
ApiRecord.objects.filter(is_active=False).update(value=0)

# DELETE
ApiRecord.objects.get(id=1).delete()
ApiRecord.objects.filter(value=0).delete() # delete many
```

C2 — Ordering, Limiting, Counting

```
# Order by value (ascending)
ApiRecord.objects.all().order_by('value')

# Order by newest first (- means descending)
ApiRecord.objects.all().order_by('-created_at')

# Get only first 10 records
ApiRecord.objects.all()[:10]

# Count how many records exist
total = ApiRecord.objects.count()

# Check if any record exists
exists = ApiRecord.objects.filter(title='Gold').exists()

# Returns True or False
```

C3 — update_or_create Explained

```

# This is what we use in fetch_data – very important to understand

obj, created = ApiRecord.objects.update_or_create(
    title='Gold',          # ← LOOKUP FIELD: find a record with this title
    defaults={             # ← UPDATE THESE FIELDS if found, or set on create
        'value': 1923.5,
        'is_active': True,
    }
)

# 'created' is True if a new record was made, False if existing was updated
if created:
    print('New record added!')
else:
    print('Existing record updated!')

```

■ **TIP:** Use `update_or_create` so you never get duplicate records. The first argument is the unique identifier — usually title or an ID from the API.

PART D — Static Files & Styling Your Website

Static files are CSS, JavaScript, and images. Django needs special setup to serve these. This was NOT covered in the main guide.

D1 — Create Static Files Folder

```
# Create this folder structure:
livedata/
├── static/
│   ├── livedata/
│   │   ├── style.css
│   │   └── script.js
```

D2 — Configure settings.py for Static Files

```
# Add to settings.py:
import os

STATIC_URL = '/static/'

# Where collectstatic puts files for production:
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')

# Where Django looks for static files during development:
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, 'livedata', 'static'),
]
```

D3 — Use Static Files in HTML Templates

```
<!-- At the TOP of every template that uses static files -->
{% load static %}

<!DOCTYPE html>
<html>
<head>
    <!-- Link your CSS file -->
    <link rel='stylesheet' href="{% static 'livedata/style.css' %}">
</head>
<body>
    <!-- Link your JS file -->
    <script src="{% static 'livedata/script.js' %}"></script>
</body>
</html>
```

D4 — Example CSS for a Nice Dashboard (style.css)

```
/* File: livedata/static/livedata/style.css */

body {
    font-family: 'Segoe UI', sans-serif;
    background: #f0f4ff;
    margin: 0;
    padding: 0;
```

```

}

.navbar { background: #1a1a2e; color: white; padding: 1rem 2rem; }
.navbar h1 { margin: 0; font-size: 1.4rem; }

.container { max-width: 1100px; margin: 2rem auto; padding: 0 1rem; }

.card {
  background: white;
  border-radius: 12px;
  padding: 1.5rem;
  box-shadow: 0 2px 12px rgba(0,0,0,0.08);
  margin-bottom: 1rem;
}

.badge-active { background:#d4edda; color:#155724; padding:3px 8px; border-radius:4px; }
.badge-inactive { background:#f8d7da; color:#721c24; padding:3px 8px; border-radius:4px; }

table { width:100%; border-collapse:collapse; }
th { background:#1a1a2e; color:white; padding:10px; text-align:left; }
td { padding:10px; border-bottom:1px solid #eee; }
tr:hover { background:#f8f9ff; }

```

■ ■ **NOTE:** After adding static files, run: `python manage.py collectstatic` — you will be asked to confirm.

PART E — Environment Variables & Production Security

The main guide only covered basic .env usage. Here is the complete picture including production-safe settings — critical before going live.

E1 — Full .env File Template

```
# .env — NEVER commit this file to Git!

# Django
SECRET_KEY=your-very-long-random-secret-key-here
DEBUG=True # Change to False in production!
ALLOWED_HOSTS=localhost,127.0.0.1

# Database
DB_NAME=api_data_db
DB_USER=api_user
DB_PASSWORD=your_strong_password
DB_HOST=localhost
DB_PORT=5432

# Your API
API_URL=https://api.example.com/v1/data
API_KEY=your_api_key_here

# Email (for error alerts — optional)
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=you@gmail.com
EMAIL_PASSWORD=your_app_password
```

E2 — Generate a Secure SECRET_KEY

```
# Run this in Python to generate a new secret key:
python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"

# Copy the output into your .env file
```

E3 — Production Settings Checklist

Setting	Development Value	Production Value	Why It Matters
DEBUG	True	False	True exposes error details to hackers
ALLOWED_HOSTS	localhost	yourdomain.com	Prevents HTTP Host header attacks
SECRET_KEY	any string	long random string	Signs cookies/sessions — must be secret
DATABASES HOST	localhost	your DB server IP	Points to real production database
HTTPS	not needed	required	Encrypts all data between user and server

PART F — Django REST Framework (Serve Your Data as an API)

DRF lets you expose your PostgreSQL data as a JSON API — so other apps or dashboards can consume it. Very useful if you want a mobile app or charts to read your data.

```
pip install djangorestframework
```

F1 — Add to settings.py

```
INSTALLED_APPS = [  
    ...  
    'rest_framework', # Add this  
    'livedata',  
]
```

F2 — Create a Serializer (livedata/serializers.py)

```
# File: livedata/serializers.py  
from rest_framework import serializers  
from .models import ApiRecord  
  
class ApiRecordSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = ApiRecord  
        fields = '__all__' # include all fields
```

F3 — Create API Views (livedata/views.py)

```
from rest_framework import generics  
from .models import ApiRecord  
from .serializers import ApiRecordSerializer  
  
# List all records as JSON  
class ApiRecordListView(generics.ListAPIView):  
    queryset = ApiRecord.objects.all()  
    serializer_class = ApiRecordSerializer  
  
# Get a single record as JSON  
class ApiRecordDetailView(generics.RetrieveAPIView):  
    queryset = ApiRecord.objects.all()  
    serializer_class = ApiRecordSerializer
```

F4 — Add API URLs

```
# In livedata/urls.py, add:  
from .views import ApiRecordListView, ApiRecordDetailView  
  
urlpatterns = [  
    ...existing urls...  
    path('api/records/', ApiRecordListView.as_view(), name='api-list'),  
    path('api/records//', ApiRecordDetailView.as_view(), name='api-detail'),  
]  
  
# Access at: http://127.0.0.1:8000/api/records/
```

PART G — Logging & Monitoring Your App

Logging means writing messages to a file when things happen — successes and errors. Without logging you are flying blind. Add this to settings.py:

```
# Add to settings.py:
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'verbose': {
            'format': '{levelname} {asctime} {module} {message}',
            'style': '{',
        },
    },
    'handlers': {
        'file': {
            'level': 'INFO',
            'class': 'logging.FileHandler',
            'filename': 'debug.log',
            'formatter': 'verbose',
        },
        'console': {
            'class': 'logging.StreamHandler',
        },
    },
    'root': {
        'handlers': ['file', 'console'],
        'level': 'INFO',
    },
}
```

G2 — Use Logging in Your Fetch Script

```
import logging
logger = logging.getLogger(__name__)

def fetch_and_save():
    logger.info('Starting API fetch...')
    try:
        response = requests.get(api_url, timeout=10)
        logger.info(f'API responded with status {response.status_code}')
        # ... save records ...
        logger.info(f'Successfully saved {count} records')
    except Exception as e:
        logger.error(f'Fetch failed: {e}', exc_info=True)
```

■ **TIP:** Check debug.log whenever something stops working. It will tell you exactly what happened and when.

PART H — Database Migrations — Deep Dive

Migrations are files that track database changes. The main guide only showed the basics. Here is everything you need to know to avoid destroying your data.

H1 — Migration Workflow Explained

Command	What It Does	When to Run
<code>makemigrations</code>	Creates migration files from your models.py changes	After EVERY change to models.py
<code>migrate</code>	Applies migration files to the actual database	After makemigrations, and on first setup
<code>showmigrations</code>	Lists all migrations and whether they've been applied	While debugging migration issues
<code>sqlmigrate app 0001</code>	Shows the raw SQL that a migration will run	When you want to understand what will change
<code>migrate app zero</code>	Rolls back ALL migrations for an app	Emergency undo — use carefully!

H2 — Adding a New Field Safely

```
# 1. Edit models.py — add the new field WITH a default:
class ApiRecord(models.Model):
    ...existing fields...
    category = models.CharField(max_length=100, default='general') # NEW

# 2. Create migration
python manage.py makemigrations

# 3. Apply it
python manage.py migrate
```

■■ **NOTE:** Always add 'default=...' or 'null=True' when adding a new field to an existing table that has data. Without this, Django cannot fill in the value for existing rows and will error.

PART I — Deploying to a Real Website (3 Options)

The main guide only ran the development server (python manage.py runserver). That is NOT a real website — it only works on your computer. Here are 3 beginner-friendly hosting options.

Option 1 — PythonAnywhere (Easiest for Beginners — FREE tier available)

- Go to <https://www.pythonanywhere.com> and create a free account
- Click **Web** tab → **Add a new web app** → Choose Django
- In the **Bash console**, clone your project: `git clone https://github.com/you/yourproject`
- Install requirements: `pip install -r requirements.txt`
- Edit the WSGI file to point to your project
- Set environment variables in the **.env** section of the dashboard
- Click **Reload** — your site is live!

```
# requirements.txt – create this file in your project:
pip freeze > requirements.txt

# This saves all your installed libraries so the server can install them too
```

Option 2 — Railway (Modern, Easy, Free Tier)

- Go to <https://railway.app> and sign in with GitHub
- Click **New Project** → **Deploy from GitHub repo**
- Add a PostgreSQL database from the Railway dashboard
- Set environment variables in the **Variables** tab
- Railway auto-detects Django and deploys automatically on every Git push

Option 3 — Traditional VPS (DigitalOcean / Hetzner)

More complex but full control. Requires setting up Nginx + Gunicorn:

```
# Install Gunicorn (production web server for Django)
pip install gunicorn

# Test it locally:
gunicorn mysite.wsgi:application

# On the server, create a service file:
# /etc/systemd/system/mysite.service
[Unit]
Description=Django App

[Service]
WorkingDirectory=/home/user/my_api_project
ExecStart=/home/user/my_api_project/venv/bin/gunicorn mysite.wsgi:application
Restart=always

[Install]
WantedBy=multi-user.target
```

■ **TIP:** For beginners: start with PythonAnywhere. It is the simplest and has a free tier perfect for learning.

PART J — FULL ERROR ENCYCLOPEDIA (60+ Errors With Fixes)

Every error below includes: the **exact message** you will see, **why it happens**, and the **exact fix**. Errors are grouped by category.

■■■ INSTALLATION & SETUP ERRORS ■■■

ERROR #1 — 'python' is not recognized

What you see in the terminal/browser:

```
python is not recognized as an internal or external command
```

Why it happens: Python was not added to the system PATH during installation.

■ **FIX:** Uninstall Python and reinstall it. On the first install screen, tick the checkbox 'Add Python to PATH' before clicking Install.

```
# Verify after reinstalling:  
python --version
```

ERROR #2 — 'pip' is not recognized

What you see in the terminal/browser:

```
pip is not recognized as an internal or external command
```

Why it happens: Same PATH issue as above, or pip is not installed.

■ **FIX:** Try: `python -m pip install django` (using `python -m pip` instead of just `pip`)

```
python -m pip install django psycpg2-binary requests python-dotenv
```

ERROR #3 — Permission denied when installing packages

What you see in the terminal/browser:

```
ERROR: Could not install packages due to an OSError: [Errno 13] Permission denied
```

Why it happens: You are trying to install packages system-wide without admin rights.

■ **FIX:** Always activate your virtual environment first. If you see (venv) in your terminal, you have permission.

```
# Activate venv first (Windows):  
venv\Scripts\activate  
# Then install:  
pip install django
```

ERROR #4 — venv activation fails on Windows

What you see in the terminal/browser:

```
cannot be loaded because running scripts is disabled on this system
```

Why it happens: Windows blocks running scripts by default (a security policy).

■ **FIX:** Open PowerShell as Administrator and run this one-time command:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

ERROR #5 — psycpg2 install fails

What you see in the terminal/browser:

```
ERROR: Failed building wheel for psycpg2
```

Why it happens: psycpg2 needs C compiler tools to build. psycpg2-binary avoids this.

■ **FIX:** Uninstall psycpg2 and install psycpg2-binary instead:

```
pip uninstall psycpg2  
pip install psycpg2-binary
```

■■■ POSTGRESQL CONNECTION ERRORS ■■■

ERROR #6 — Connection refused

What you see in the terminal/browser:

```
django.db.utils.OperationalError: could not connect to server: Connection refused Is the server running on host 'localhost' and accepting TCP/IP connections on port 5432?
```

Why it happens: PostgreSQL service is not running.

■ **FIX:** Start the PostgreSQL service:

```
# Windows: Open Services app, find 'postgresql-x64-XX', click Start
# Mac:
brew services start postgresql
# Linux:
sudo systemctl start postgresql
```

ERROR #7 — Password authentication failed

What you see in the terminal/browser:

```
django.db.utils.OperationalError: FATAL: password authentication failed for user 'api_user'
```

Why it happens: Wrong password in your .env file, or the user does not exist.

■ **FIX:** Check your .env DB_PASSWORD. To reset in psql:

```
psql -U postgres
ALTER USER api_user WITH PASSWORD 'new_password';
\q
# Then update .env with the new password
```

ERROR #8 — Database does not exist

What you see in the terminal/browser:

```
django.db.utils.OperationalError: FATAL: database 'api_data_db' does not exist
```

Why it happens: The database was never created, or the name in .env is wrong.

■ **FIX:** Create it:

```
psql -U postgres
CREATE DATABASE api_data_db;
\q
```

ERROR #9 — Role does not exist

What you see in the terminal/browser:

```
FATAL: role 'api_user' does not exist
```

Why it happens: The PostgreSQL user was not created.

■ **FIX:** Create the user:

```
psql -U postgres
CREATE USER api_user WITH PASSWORD 'your_password';
GRANT ALL PRIVILEGES ON DATABASE api_data_db TO api_user;
\q
```

ERROR #10 — No pg_hba.conf entry

What you see in the terminal/browser:

```
FATAL: no pg_hba.conf entry for host '127.0.0.1', user 'api_user'
```

Why it happens: PostgreSQL's security file does not allow your user to connect.

■ **FIX:** Edit pg_hba.conf (usually at /etc/postgresql/*/main/pg_hba.conf on Linux) and add:

```
# Add this line near the bottom of pg_hba.conf:  
local all api_user md5  
# Then restart PostgreSQL:  
sudo systemctl restart postgresql
```

■■■ DJANGO SETUP ERRORS ■■■

ERROR #11 — ImproperlyConfigured — SECRET_KEY

What you see in the terminal/browser:

`django.core.exceptions.ImproperlyConfigured: The SECRET_KEY setting must not be empty`

Why it happens: .env file is not being loaded, or SECRET_KEY is missing from it.

■ **FIX:** Make sure `load_dotenv()` is at the top of `settings.py`, before any `os.getenv()` calls.

```
# First lines of settings.py must be:
import os
from dotenv import load_dotenv
load_dotenv()

SECRET_KEY = os.getenv('SECRET_KEY')
```

ERROR #12 — ModuleNotFoundError — django

What you see in the terminal/browser:

`ModuleNotFoundError: No module named 'django'`

Why it happens: Virtual environment is not activated, or Django is not installed in it.

■ **FIX:** Activate venv first, then check:

```
# Activate:
venv\Scripts\activate # Windows
source venv/bin/activate # Mac/Linux

# Verify Django is installed:
pip show django
```

ERROR #13 — ModuleNotFoundError — livedata

What you see in the terminal/browser:

`ModuleNotFoundError: No module named 'livedata'`

Why it happens: 'livedata' is not in `INSTALLED_APPS` in `settings.py`.

■ **FIX:** Add 'livedata' to `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    ...
    'livedata', # Add this!
]
```

ERROR #14 — App has no migrations

What you see in the terminal/browser:

`CommandError: App 'livedata' does not have migrations`

Why it happens: You never ran `makemigrations` for the app, or the migrations folder is missing.

■ **FIX:** Run:

```
python manage.py makemigrations livedata
python manage.py migrate
```

ERROR #15 — Table does not exist

What you see in the terminal/browser:

`django.db.utils.ProgrammingError: relation 'livedata_apirecord' does not exist`

Why it happens: Migrations were created but not applied to the database.

■ **FIX:** Apply the migrations:

```
python manage.py migrate
```

ERROR #16 — No module named 'dotenv'

What you see in the terminal/browser:

```
ModuleNotFoundError: No module named 'dotenv'
```

Why it happens: python-dotenv was not installed.

■ **FIX:** Install it:

```
pip install python-dotenv
```

■■■ DJANGO URL & VIEW ERRORS ■■■

ERROR #17 — Page not found 404

What you see in the terminal/browser:

Page not found (404) Using the URLconf defined in mysite.urls, Django tried these URL patterns, in this order: ...

Why it happens: The URL you visited does not match any pattern in urls.py.

■ **FIX:** Check your urls.py files. Make sure livedata.urls is included in mysite/urls.py:

```
# mysite/urls.py
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('livedata.urls')), # This line must exist!
]
```

ERROR #18 — TemplateDoesNotExist

What you see in the terminal/browser:

django.template.exceptions.TemplateDoesNotExist: livedata/index.html

Why it happens: Template file is in the wrong folder, or the folder structure is wrong.

■ **FIX:** The file must be at EXACTLY: livedata/templates/livedata/index.html (two levels of 'livedata')

```
# Correct folder structure:
livedata/
  templates/
    livedata/
      index.html ← must be HERE
      base.html
```

ERROR #19 — NoReverseMatch

What you see in the terminal/browser:

django.urls.exceptions.NoReverseMatch: Reverse for 'index' not found

Why it happens: You used {% url 'index' %} in a template but the URL name does not exist.

■ **FIX:** Check that the URL name in urls.py matches exactly what you use in the template:

```
# In urls.py:
path('', views.index, name='index') # name='index'

# In template:
<a href="{% url 'index' %}">Home</a>
```

ERROR #20 — AttributeError in template

What you see in the terminal/browser:

AttributeError: 'QuerySet' object has no attribute 'title'

Why it happens: You passed the whole QuerySet to the template but tried to access a field directly.

■ **FIX:** Pass individual objects. In the view, either pass a queryset (loop in template) or a single object:

```
# For a list — pass queryset:
return render(request, 'index.html', {'records': ApiRecord.objects.all()})
# In template: {% for r in records %}{{ r.title }}{% endfor %}

# For single object:
record = ApiRecord.objects.get(pk=pk)
return render(request, 'detail.html', {'record': record})
# In template: {{ record.title }}
```


■■■ API FETCHING ERRORS ■■■

ERROR #21 — ConnectionError — API

What you see in the terminal/browser:

```
requests.exceptions.ConnectionError: HTTPSConnectionPool(host='api.example.com', port=443): Max retries exceeded
```

Why it happens: The API URL is wrong, or there is no internet connection.

■ **FIX:** Check the API URL in your .env file. Test it by opening it in a browser first.

```
# Debug:
import requests
print(os.getenv('API_URL')) # Make sure this prints the correct URL
response = requests.get(os.getenv('API_URL'))
print(response.status_code) # Should be 200
```

ERROR #22 — 401 Unauthorized

What you see in the terminal/browser:

```
requests.exceptions.HTTPError: 401 Client Error: Unauthorized for url
```

Why it happens: Your API key is wrong or missing.

■ **FIX:** Check the API_KEY in .env and how the API expects it (header vs query param):

```
# Check what the API returns:
print(response.status_code) # 401 means auth failed
print(response.text) # Often tells you exactly what's wrong
```

ERROR #23 — 403 Forbidden

What you see in the terminal/browser:

```
requests.exceptions.HTTPError: 403 Client Error: Forbidden
```

Why it happens: Your API key is valid but your plan does not have access to that endpoint.

■ **FIX:** Check your API account dashboard. You may need to upgrade your plan or use a different endpoint.

ERROR #24 — 429 Too Many Requests

What you see in the terminal/browser:

```
requests.exceptions.HTTPError: 429 Client Error: Too Many Requests
```

Why it happens: You are calling the API too frequently and hit the rate limit.

■ **FIX:** Add a delay between requests and reduce fetch frequency:

```
import time
for item in records:
    # process item
    time.sleep(0.5) # wait 0.5 seconds between requests

# Also reduce your cron schedule from */1 to */5 (every 5 min)
```

ERROR #25 — JSONDecodeError

What you see in the terminal/browser:

```
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

Why it happens: The API returned HTML (like an error page) instead of JSON.

■ **FIX:** Print the raw response to see what the API actually returned:

```
response = requests.get(api_url)
print('Status:', response.status_code)
print('Content-Type:', response.headers.get('Content-Type'))
```

```
print('Body:', response.text[:300]) # Print first 300 characters
```

ERROR #26 — SSL Certificate Error

What you see in the terminal/browser:

```
requests.exceptions.SSLError: HTTPSConnectionPool: certificate verify failed
```

Why it happens: The API's SSL certificate cannot be verified (common with local/dev APIs).

■ **FIX:** For development only — add `verify=False` (do NOT use in production):

```
# Development only! Never do this in production:
response = requests.get(api_url, verify=False)

# Better fix: install certifi
pip install certifi
```

ERROR #27 — Timeout Error

What you see in the terminal/browser:

```
requests.exceptions.ReadTimeout: HTTPSConnectionPool: Read timed out.
```

Why it happens: The API took too long to respond.

■ **FIX:** Increase timeout or add a retry:

```
# Increase timeout (seconds):
response = requests.get(api_url, timeout=30)

# Or add retry logic:
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()
retry = Retry(total=3, backoff_factor=1)
session.mount('https://', HTTPAdapter(max_retries=retry))
response = session.get(api_url, timeout=30)
```

■■■ DATABASE SAVE ERRORS ■■■

ERROR #28 — IntegrityError — duplicate key

What you see in the terminal/browser:

```
django.db.utils.IntegrityError: duplicate key value violates unique constraint
```

Why it happens: You tried to save a record that already exists with a unique field value.

■ **FIX:** Use `update_or_create` instead of `create`:

```
# WRONG:
ApiRecord.objects.create(title='Gold', value=1923)

# RIGHT (handles duplicates automatically):
ApiRecord.objects.update_or_create(
    title='Gold',          # lookup field
    defaults={'value': 1923} # update this if found
)
```

ERROR #29 — DataError — value too long

What you see in the terminal/browser:

```
django.db.utils.DataError: value too long for type character varying(255)
```

Why it happens: The API returned a string longer than your CharField's `max_length`.

■ **FIX:** Increase `max_length` in `models.py`, then run migrations:

```
# In models.py, increase max_length:
title = models.CharField(max_length=500) # was 255

# Then:
python manage.py makemigrations
python manage.py migrate
```

ERROR #30 — ValueError — null into non-null field

What you see in the terminal/browser:

```
django.db.utils.IntegrityError: null value in column 'value' violates not-null constraint
```

Why it happens: The API returned `None`/null for a field that your model requires.

■ **FIX:** Add `null=True` and `blank=True` to the field in `models.py`:

```
# In models.py:
value = models.FloatField(null=True, blank=True) # Allow null

# Then migrate:
python manage.py makemigrations
python manage.py migrate
```

ERROR #31 — TypeError — wrong field type

What you see in the terminal/browser:

```
TypeError: Field 'value' expected a number but got 'N/A'
```

Why it happens: The API returned a string where you expected a number.

■ **FIX:** Convert or default the value before saving:

```
# Safe conversion:
try:
    val = float(item.get('value', 0))
except (TypeError, ValueError):
    val = None
```

```
# Then save:
ApiRecord.objects.update_or_create(
    title=item.get('title'),
    defaults={'value': val}
)
```

■■■ MIGRATION ERRORS ■■■

ERROR #32 — Migration conflicts

What you see in the terminal/browser:

```
CommandError: Conflicting migrations detected; multiple leaf nodes in the migration graph
```

Why it happens: Two migration files were created at the same time (common in teams).

■ **FIX:** Merge the migrations:

```
python manage.py makemigrations --merge
python manage.py migrate
```

ERROR #33 — Fake migration needed

What you see in the terminal/browser:

```
django.db.utils.ProgrammingError: relation already exists
```

Why it happens: The table exists in the DB but Django thinks the migration hasn't run.

■ **FIX:** Fake the migration to tell Django it is done:

```
python manage.py migrate --fake livedata 0001
```

ERROR #34 — Cannot add column with no default

What you see in the terminal/browser:

```
django.db.utils.IntegrityError: column 'new_field' of relation has no default
```

Why it happens: Adding a required field to a table that already has rows of data.

■ **FIX:** Always provide a default when adding new fields to existing models:

```
# In models.py, always add default:
new_field = models.CharField(max_length=100, default='unknown')
# OR allow null:
new_field = models.CharField(max_length=100, null=True, blank=True)
```

■■■ TEMPLATE & STATIC FILE ERRORS ■■■

ERROR #35 — Static files not loading (404 in browser console)

What you see in the terminal/browser:

```
GET http://127.0.0.1:8000/static/livedata/style.css 404 (Not Found)
```

Why it happens: Static files are not configured or `{% load static %}` is missing from the template.

■ **FIX:** 1. Add `{% load static %}` at top of template. 2. Check `STATIC_URL` in settings.py. 3. Make sure file is in correct folder:

```
# settings.py must have:
STATIC_URL = '/static/'

# Template must start with:
{% load static %}

# File must be at:
livedata/static/livedata/style.css
```

ERROR #36 — Template variable shows nothing

What you see in the terminal/browser:

```
The template renders but {{ record.title }} shows blank
```

Why it happens: Variable name mismatch between views.py and template, or data is not being passed.

■ **FIX:** Check the variable name in `render()` matches what you use in the template:

```
# views.py:
return render(request, 'index.html', {'records': data}) # key is 'records'

# template — use the SAME key:
{% for r in records %}{{ r.title }}{% endfor %}
```

ERROR #37 — Unclosed block tag

What you see in the terminal/browser:

```
TemplateSyntaxError: Unclosed block tag on line X: 'for'. Looking for one of: 'empty',
'endfor'
```

Why it happens: Every `{% for %}` or `{% if %}` must be closed with `{% endfor %}` or `{% endif %}`.

■ **FIX:** Find the unclosed tag on the line mentioned and close it:

```
{% for r in records %}
    <tr><td>{{ r.title }}</td></tr>
{% endfor %} <!-- Don't forget this! -->
```

■■■ ENVIRONMENT & SETTINGS ERRORS ■■■

ERROR #38 — ALLOWED_HOSTS error in production

What you see in the terminal/browser:

```
django.core.exceptions.DisallowedHost: Invalid HTTP_HOST header: 'yourdomain.com'
```

Why it happens: Your domain is not in ALLOWED_HOSTS in settings.py.

■ **FIX:** Add your domain to ALLOWED_HOSTS:

```
# settings.py:
ALLOWED_HOSTS = ['yourdomain.com', 'www.yourdomain.com', 'localhost']
```

ERROR #39 — .env values not loading

What you see in the terminal/browser:

```
None is returned from os.getenv('DB_NAME') — DB_NAME is None
```

Why it happens: load_dotenv() was not called before os.getenv(), or .env file has spaces around '='.

■ **FIX:** 1. Ensure load_dotenv() is at the TOP of settings.py. 2. Check .env has no spaces around =:

```
# WRONG in .env:
DB_NAME = api_data_db

# RIGHT in .env:
DB_NAME=api_data_db

# WRONG in settings.py (too late):
some_code = 'blah'
load_dotenv() # Too late!

# RIGHT:
import os
from dotenv import load_dotenv
load_dotenv() # Must be FIRST
```

ERROR #40 — SECRET_KEY changed in production

What you see in the terminal/browser:

```
All existing sessions are invalidated. Users get logged out. CSRF errors appear.
```

Why it happens: You regenerated the SECRET_KEY and existing cookies/sessions are now invalid.

■ **FIX:** Never change SECRET_KEY on a running production server. Keep it in .env and never regenerate it once set.

■■■ SCHEDULING & AUTOMATION ERRORS ■■■

ERROR #41 — Cron not running the script

What you see in the terminal/browser:

```
Script runs manually but nothing happens when cron fires
```

Why it happens: Cron uses an empty environment — it does not know where Python or your project are.

■ **FIX:** Use absolute paths in your cron job:

```
# WRONG:
*/5 * * * * python manage.py fetch_api_data

# RIGHT (use full paths):
*/5 * * * * /home/user/my_api_project/venv/bin/python /home/user/my_api_project/manage.py fetch_api_data >> /home/user/cron.log 2>&1
```

ERROR #42 — Cron runs but .env not loaded

What you see in the terminal/browser:

Fetch runs but all DB values are None – database connection fails

Why it happens: Cron does not read your .env file automatically.

■ **FIX:** Either hardcode environment variables in crontab or source .env in a wrapper script:

```
# Option 1: Source .env in cron:
*/5 * * * * cd /home/user/my_api_project && source .env && ./venv/bin/python manage.py fetch_a
pi_data

# Option 2: Add env vars directly to crontab:
*/5 * * * * DB_NAME=api_data_db DB_PASSWORD=secret /path/to/venv/bin/python manage.py fetch_ap
i_data
```

■■■ SERVER & DEPLOYMENT ERRORS ■■■

ERROR #43 — Static files missing in production

What you see in the terminal/browser:

CSS and images don't load on the live server – 404 errors in browser console

Why it happens: In production (DEBUG=False), Django doesn't serve static files. You must collect them.

■ **FIX:** Run collectstatic and configure your web server to serve the staticfiles/ folder:

```
# Run this before deploying:
python manage.py collectstatic

# On PythonAnywhere: set Static files URL to /static/ and
# directory to /home/user/my_api_project/staticfiles/
```

ERROR #44 — 500 Internal Server Error in production

What you see in the terminal/browser:

500 Internal Server Error – no details shown

Why it happens: DEBUG=False hides error details. The error is in the server log.

■ **FIX:** Check the error log on your hosting platform:

```
# PythonAnywhere: Web tab → Error log
# Railway: Deployments → View logs
# VPS: sudo journalctl -u mysite.service -n 50

# Temporarily set DEBUG=True to see the error, then set back to False
```

ERROR #45 — Port already in use

What you see in the terminal/browser:

Error: That port is already in use.

Why it happens: Another process is using port 8000 (maybe a previous runserver you didn't stop).

■ **FIX:** Kill the existing process:

```
# Find and kill process on port 8000:
# Windows:
netstat -ano | findstr :8000
taskkill /PID /F

# Mac/Linux:
lsof -i :8000
kill -9

# Or run on a different port:
python manage.py runserver 8080
```

■■■ PYTHON & GENERAL ERRORS ■■■

ERROR #46 — IndentationError

What you see in the terminal/browser:

IndentationError: unexpected indent / IndentationError: expected an indented block

Why it happens: Python requires consistent indentation. Mixing tabs and spaces causes this error.

■ **FIX:** Use only spaces (4 spaces per level). In VS Code: Edit → Convert Indentation to Spaces

```
# WRONG (mixed tabs and spaces):
def my_func():
```

```

    print('hello') # Tab then spaces

# RIGHT (4 spaces only):
def my_func():
    print('hello') # 4 spaces

```

ERROR #47 — SyntaxError — invalid syntax

What you see in the terminal/browser:

```
SyntaxError: invalid syntax (points to a line)
```

Why it happens: A typo in your Python code — missing colon, bracket, or quote.

■ **FIX:** Look at the line pointed to AND the line above it (the error is often there):

```

# Common causes:
# Missing colon:
def my_func() # WRONG
def my_func(): # RIGHT

# Unclosed bracket:
my_dict = {'key': 'value' # WRONG (missing }}
my_dict = {'key': 'value'} # RIGHT

```

ERROR #48 — KeyError

What you see in the terminal/browser:

```
KeyError: 'title'
```

Why it happens: You tried to access a dictionary key that doesn't exist in the API response.

■ **FIX:** Use `.get()` instead of `[]` to provide a default value:

```

# WRONG:
title = item['title'] # crashes if 'title' not in response

# RIGHT:
title = item.get('title', 'No Title') # returns 'No Title' if missing

```

ERROR #49 — AttributeError — NoneType

What you see in the terminal/browser:

```
AttributeError: 'NoneType' object has no attribute 'get'
```

Why it happens: A variable you expected to have data is None (empty).

■ **FIX:** Add a check before using the variable:

```

# Check for None before using:
data = response.json()
if data is None:
    print('API returned nothing!')
    return

# Or use 'or' for default:
records = data.get('results') or [] # empty list if None

```

ERROR #50 — RecursionError

What you see in the terminal/browser:

```
RecursionError: maximum recursion depth exceeded
```

Why it happens: A function is calling itself endlessly (rare but confusing when it happens).

■ **FIX:** You likely have a circular import. Check that `views.py` is not importing from `urls.py` and vice versa.

■■■ BROWSER & FRONTEND ERRORS ■■■

ERROR #51 — CSRF Verification Failed

What you see in the terminal/browser:

CSRF verification failed. Request aborted. 403 Forbidden

Why it happens: Django's security system rejected a form submission because the CSRF token is missing.

■ **FIX:** Add {% csrf_token %} inside every HTML form:

```
<form method='POST' action='/>
    {% csrf_token %}    <!-- This line is required! -->
    <input type='text' name='search'>
    <button type='submit'>Search</button>
</form>
```

ERROR #52 — CORS Error (when using DRF as API)

What you see in the terminal/browser:

Access to XMLHttpRequest blocked by CORS policy: No 'Access-Control-Allow-Origin' header

Why it happens: A frontend app (React, etc.) is trying to call your Django API from a different domain.

■ **FIX:** Install and configure django-cors-headers:

```
pip install django-cors-headers

# settings.py:
INSTALLED_APPS = [..., 'corsheaders']
MIDDLEWARE = ['corsheaders.middleware.CorsMiddleware', ...rest]
CORS_ALLOW_ALL_ORIGINS = True # Development only
# Production:
CORS_ALLOWED_ORIGINS = ['https://yourfrontend.com']
```

ERROR #53 — CSS not updating after changes

What you see in the terminal/browser:

You changed style.css but the browser still shows the old styles

Why it happens: The browser cached the old CSS file.

■ **FIX:** Hard refresh the browser: Ctrl+Shift+R (Windows) or Cmd+Shift+R (Mac). Or open DevTools → Network → check 'Disable cache'.

■■■ ADMIN PANEL ERRORS ■■■

ERROR #54 — Admin page shows no data / model not visible

What you see in the terminal/browser:

You go to /admin/ but your ApiRecord model is not listed

Why it happens: The model was not registered in admin.py.

■ **FIX:** Register it:

```
# livedata/admin.py:
from django.contrib import admin
from .models import ApiRecord

admin.site.register(ApiRecord)
```

ERROR #55 — Admin login doesn't work

What you see in the terminal/browser:

Please enter the correct username and password. Note that both fields may be case-sensitive.

Why it happens: Wrong credentials or superuser was never created.

■ **FIX:** Create a new superuser:

```
python manage.py createsuperuser
# Follow the prompts. Username is case-sensitive!
```

■■■ WINDOWS-SPECIFIC ERRORS ■■■

ERROR #56 — Long path error on Windows

What you see in the terminal/browser:

FileNotFoundError: [WinError 3] The system cannot find the path specified (very long path)

Why it happens: Windows has a 260-character path limit by default.

■ **FIX:** Enable long paths in Windows: run regedit → HKLM\SYSTEM\CurrentControlSet\Control\FileSystem → set LongPathsEnabled to 1. Or move project to a shorter path like C:\proj\

ERROR #57 — \n line endings causing issues

What you see in the terminal/browser:

Syntax errors in .env or Python files that look correct

Why it happens: Windows saves files with CRLF line endings (\r\n) but some tools expect LF (\n).

■ **FIX:** In VS Code, check bottom-right corner shows 'LF' not 'CRLF'. Click it to change. Or install git with 'Checkout as-is, commit Unix-style' option.

■■■ PERFORMANCE & DATA ERRORS ■■■

ERROR #58 — Too many database queries (slow page)

What you see in the terminal/browser:

Page loads slowly, Django debug toolbar shows 100+ queries

Why it happens: Accessing related data inside a loop causes N+1 query problem.

■ **FIX:** Use select_related or prefetch_related:

```
# Instead of:
records = ApiRecord.objects.all()
# for r in records: r.category.name ← extra query per record!

# Use:
records = ApiRecord.objects.all().select_related('category')
```

ERROR #59 — Memory error with large API responses

What you see in the terminal/browser:

MemoryError when processing API response with thousands of records

Why it happens: Loading the entire API response into memory at once.

■ **FIX:** Process in batches:

```
# Process 100 records at a time:
batch_size = 100
for i in range(0, len(records), batch_size):
    batch = records[i:i+batch_size]
    for item in batch:
        ApiRecord.objects.update_or_create(...)
    print(f'Processed batch {i//batch_size + 1}')
```

ERROR #60 — Timezone issues — wrong timestamps

What you see in the terminal/browser:

Timestamps saved to DB are off by several hours from your local time

Why it happens: Django defaults to UTC timezone but you are in a different timezone (Dubai is UTC+4).

■ **FIX:** Set timezone in settings.py:

```
# settings.py:
TIME_ZONE = 'Asia/Dubai' # Your timezone
USE_TZ = True # Always keep this True

# In templates, use Django's date filter:
{{ record.created_at|date:'d M Y H:i' }}
```

■ Quick Reference — Error First Aid

When something breaks, follow this sequence:

1. READ THE ERROR	Don't panic. Read the full error message. The last few lines tell you exactly what went wrong and which file/line.
2. GOOGLE THE ERROR	Copy the key part of the error (not your file path) and paste into Google. Add 'Django' or 'Python' to the search. Stack Overflow almost always has the answer.
3. CHECK THE LOG	Look at debug.log in your project folder. Also check the terminal where runserver is running — errors print there.
4. ADD PRINT STATEMENTS	Before the broken line, add: <code>print('I reached here', variable_name)</code> — this tells you what data you have at that point.
5. UNDO & RETRY	Use git to go back to the last working state: <code>git stash</code> — then reapply changes carefully.

You now have everything you need — both guides together cover the complete picture from zero to a live website. Follow the main guide step by step, and refer back to this supplement whenever you hit a topic that needs more depth or when you encounter any error. Good luck! ■